Grouping and Analyzing of Darknet Scanners with Graph Embeddings

Nealson Setiawan
nealsons@usc.edu

ACM Reference Format:
Nealson Setiawan, nealsons@usc.edu, . 2026. Grouping and Analyzing of Darknet Scanners with Graph Embeddings. In . ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 WEEKLY NOTES

https://docs.google.com/document/d/1yYlWUAgZjqWc5p1Iukg2Bnkx2LSGkQsHafJwX5RD4Z0/edit?usp=sharing

2 RESEARCH PROBLEM STATEMENT

The analyzing and interpretation of scanners' behavior in Darknet data based on its scanning statistics using clustering on graph-ML embeddings.

3 INTRODUCTION

The Darknet is an "unused portion of a globally routed /9 and /10 IPv4 network (UCSD CAIDA Definition of the darknet)". Due to the way the network is setup, the traffic that reaches here come from various reasons such as misconfigurations of web services, malicious activities (scanning, probing, worms, etc), and other reasons. A majority of the packets sent here are from scanners. Scanners are devices that send packets broadly (or horizontally) to various number of ip addresses and/or ports to determine whether a service is hosted at a particular network. From doing so, scanners develop a fingerprint of the internet and its services. This can be used for cybersecurity evaluation of a service as well as malicious attacks. As such, scanners that sends packets into the darknet, though while not inherently malicious, can be seen as a suspicious action. Thus, understanding scanners behavior in the darknet remains a hot area of interest as this provides good insight into defending/detecting attacks (in the form of malicious scans) in the future.

For this project, the goal is to introduce a new novel method using graph machine learning techniques to analyze scanner activity in the darknet and interpret the behavior of these scanners to determine the scanner's scanning strategy like horizontal scanning and scans from conficker worms. To elaborate further, Darknet data, which mostly consists of flows of ip addresses sending packets to an ip address's port, is represented in a latent space by embedding the flows using a graph neural network. By doing so, this feature representation of the network flows would be used for downstream applications like analysis or detection. Other work in analyzing scanner behavior (as shown below in the Related Works section) uses autoencoders or word embeddings to project the data into a latent space. However, since the darknet consists of packets being sent from one ip to another, the idea of representing this data as a graph neural network seemed like an appropriate representation of the data. The results of this research project would be to show that graph-based methods can be used for downstream clustering. Thus, this research project could hopefully invite further research into using graph-based methods for scanner behavior analysis.

Personally, this project is very interesting to me for two reasons: I get to see how a scanner in the darknet works, and I get to explore graph neural networks. First, scanners can be for the goodness of services or its harm. Thus, I became intrigued into figuring out why scanners are sending their packets there. I know it's probably difficult to decipher the intent of a scanner, but its behavior could imply certain intent. Second, graph-based neural networks are basically just a constrained convolutional neural network. But, I've never experimented with it before. So, this would give me a good opportunity to test it out.

4 RELATED WORK

4.1 Detecting and Interpreting Changes in Scanning Behavior in Large Network Telescopes

This paper aims to detect and analyze scanning behavior of devices scanning the darknet, which is similar to this project's goal. To do this, the paper introduces a novel two-step process: clustering on a feature representation of the data from an auto-encoder (a neural network that is trained on itself to learn a low dimensional-space representation of its training data), and then use optimal transport theory and Wasserstein distance to compare clustering distributions of consecutive days. The features they used were traffic volumes, scanning strategy, protocol type, and device scanner features like time to live values, or operating system. By doing so, the paper was able to analyze how different groupings of darknet scanner behavior evolves over the course of few days. The auto-encoder feature representation technique is evaluated against a simple PCA dimension reduction technique and shows increased jaccard score on synthetic data. This 2-stage process is evaluated on real-world datasets like the September 2016 Mirai dataset, where it successfully detected key events in Mirai's botnet activity.

This paper relates to my project as it showcases that deep learning methods can be applied to darknet data for downstream applications like clustering. With a similar research problem statement, except the behavior evolution aspect, it inspires this project to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

extend this research by trying a different method for feature representation: graph-based encoders. Finally, the paper's feature set is a good guideline for my own research.

4.2 Inferring and Investigating IoT-Generated Scanning Campaigns Targeting a Large Network Telescope

This paper aims to analyze compromised IoT devices scanning behavior in darknet data, specifically into three categories: range scans, strobe scans, and wide scans. The dataset they used is the CAIDA UCSD dataset, which is the one used in this paper as well. Their definition of compromised IoT devices are IoT devices who sent TCP-SYN packets to the darknet that sent more than 10 packets. The thought process behind this is that compromised commercial IoT devices like ip-cameras/webcams, routers, printers, and others should not be sending TCP-SYN packets to the darknet. These commercial IoT devices were collected from Shodan, and resulted in 6,797 devices found. The features they used include number of active intervals, per hour packet rate, per destination address packet rate, and other statistics. After this, DBSCAN is used to detect clusters in scanning behavior and is then matched with "scanning objectives" like range scans, strobe scans, and wide scans based on their behavior. For each class, statistics like device distribution was displayed along with other statistics like number of ports scanned, and etc.

This paper gave me a lot of insight in the analysis part of the scanning behavior. Also, this paper uses the same dataset, so its feature set and its clustering technique (especially with the parameters it uses) is definitely a great basis for my project. Even though this doesn't use any complex feature representation, this shows that without feature representation still allows for great results.

4.3 DarkVec: Automatic Analysis of Darknet Traffic with Word Embeddings

The paper aims to analyze darknet scanners by representing sequences of source IPs using NLP-based word embeddings. For this research, they captured their own darknet data by deploying their own darknet on March 2021 that captured flows of number of source IPs, number of packets, number of ports, and activity patterns. Additionally, they created their ground truth of the data on the last day of their darknet to classify the sender of the packets by looking for payload signatures. They labelled only the last day due to the sheer amount of data captured. The labels included Mirai, Censys, etc. From these labels, they trained a simple k-NN classifier as a baseline to poor results and compared it to their DarkVec methodology. Their methodology includes extracting windows of source IPs and using a Word2Vec model to produce embeddings in which KNN-graph clustering was performed. The results showed that they achieved around 96% accuracy on their ground truth labels.

Similarly, this paper embeds the darknet data into a high-dimensional embedding space and perform clustering on it. This really provide credibility into the methodology that I'm using where instead of treating the source IPs as words, the source IPs are treated as nodes in a graph. Furthermore, the ground truth that they created was really intriguing as a ground truth would've allowed much more improvement towards the analysis of my research. The DarkVec team did publish their code and data online, but upon inspection of the ground truth, it was incomplete. Their paper didn't go deep into their methodology on creating the labels but they did mention looking into the payload of the darknet packets. But, I didn't have individual packets and only flows, and after I found that the source IPs in my data didn't match theirs. Thus, I couldn't use their ground truth labels.

4.4 Poster: Shedding Light Into the Darknet: Scanning Characterization and Detection of Temporal Changes

This paper aims to automatically classify scanner behavior using clustering on a latent space projection of the darknet flow data. Similar to paper [1], this paper uses an autoencoder to generate a high-dimensional feature vector for each source IP and clusters the embeddings to see groupings. For each source IP, statistics like number of packets, scanning rate, and other things are recorded. Then, the paper uses Earth Mover's Distance to do hypothesis testing between two clusters in two different points of time to check if cluster behavior changes over time. If Earth Mover's Distance is over a threshold, then the cluster is flagged to be checked. This method is evaluated on Mozi botnet data (September 5th and 6th 2020) and a large EMD spike was noticed. Other events were noted as well.

Similarly to my paper, this paper projects netflows into a latent space and clusters them into groupings for further analysis. However, the paper uses EMD thresholding to measure dissimilarity between two cluster's distributions over time. If there were enough time, I would've tested this approach further.

5 GOALS FROM PROJECT B AND A, AND DETAILS ON THE PROGRESS

From Project A and B, these are the goals that I have completed:

(1) Gained Access to the CAIDA Darknet dataset
(2) Cleaned the data
(3) Constructed the graph and generated embeddings
(4) Explored different clustering methods and optimized for silhouette score
(5) Analyzed and interpreted each clustering behavior

From Project A and B, these are the goals that were changed/incomplete:

(1) Changed research problem statement from detection of malicious scanners to analyzing behavior of scanners
    (a) After meeting with the Professor, I was advised to change the topic to analyzing instead of detection due to the difficulty of finding ground truth for scanners that are "malicious"
(2) Used IPs that were in Shodan database as ground truth for my project for malcious scanner detection
    (a) I spent a week doing this and decided against it after meeting with my professor. I took inspiration from paper [2] where they set "compromised" IoT as IoT devices like printers, CCTVs, and other devices that scans the darknet. The device type can be determined using Shodan. However, this wasn't really greatly supported by Professor and so I dropped it.

(3) Testing different points in time and comparing similarity between clusters in different points of time similar to paper [7]
   (a) Honestly, I kept refining the graph structure and clustering that I ran out of time.
(4) Applying methodology to Mirai botnet darknet data (September 14th 2016) to see if methodology can be applied to find a Mirai clustering
   (a) After trying to get the darknet data for September 14th 2016, I realized I needed different access that data. I didn't have enough time to wait for the access. I opted to see for Conficker worms instead.

Additional goals that were added after Project A and B include:

(1) Checking for conflicker worms in the dataset
   (a) I found that the results yield a conficker cluster (without explicit clustering for it), thus, I made it a goal to research and analyze conficker behavior

6 DISCUSSION OF WORK

6.1 Goal

The goal of this project is:

(1) Construct a graph of darknet data
(2) Generate embeddings of the source nodes in the graph
(3) Cluster the feature representation
(4) Analyze and Interpret the clusters for scanning results

6.2 Data Collection

The dataset used in this paper is the same dataset used in paper [2]: UCSD CAIDA Darknet dataset [6]. This dataset uses a network telescope to capture **horizontal** scanners sending darknet traffic to their darknet destination ports. This dataset contains specific features aggregated hourly such as (as copied from the CAIDA index file [6]):

Hourly files: All hourly statistics are collected hourly for each (source IP, destination port, protocol) in a single gzip'ed file with the following entries, separated by a |

- Source IP
- Number of destination ip addresses scanned
- Approximate number of flows: this should be the number of observed unique (destination_ip, source_port, protocol) pairs
- Number of packets sent
- Timestamp of first activity (within the hour)
- Timestamp of last activity (within the hour)
- Are all packets the same size (yes: 1, no: 0)
- Average packet size
- Number of B {B | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {C | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {D | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- Number of /24 blocks scanned
- Number of non-Conficker destinations scanned
- Destination Port
- Protocol

Each instance of this data is a network flow statistic of a unique (source IP, protocol, and destination port), so each of these features describe a unique flow between source IP and destination port with a certain protocol. This separation is useful as it captures lots of different scanning behaviors. This dataset contains terrabytes of data scanned hourly from 2008 to 2017. Because of the large amount of data, this project only consists of one hour chosen arbitrarily: May 25th 2016, 11PM (GMT). While other dates like September 14th 2016 correlates with the Mirai botnet would be useful to analyze, I couldn't get access in time. Even one hour contains around 130,000 unique flows, thus adding another hour would overload the model, as this data already caused some Out of memory issues.

The URL for this dataset is: https://www.caida.org/projects/network_telescope/

6.3 Methodology

This project currently is done in three steps: processing and explanatory data analysis, building the graph and training a MetaPath2Vec model, and cluster/analyze groupings. Initially, during initial iterations of the graph building, a simple bi-partite graph was constructed and thus Node2Vec was used. But during later iterations like the final graph described below, the differentiation of types of nodes (source IPs vs destination Ports) requires heterogenous graphs construction instead. Thus, MetaPath2Vec was used which was created by Meta as a Node2Vec on heterogenous graphs. All scripts are done in Google Colab due to ease of use for GPU training needed for MetaPath2Vec. A single A100 GPU on Colab's own linux environment was used to train.

Processing The data is an hour of darknet traffic captured on May 25th 2016, 11PM (GMT). Certain features are created from other columns: activity time taken, and number of scanned ips / time taken. Next, instances where less than 10 number of ip scanned are removed similar to [2]. Finally, the numerical features are normalized using Min Max Scaling to stabilize training.

Graph representation and embedding The graph is constructed as a hetereogenous graph (graph with multiple types of nodes and edges) with two types of nodes: source IP and destination port, and one type of edge: scans. The graph is undirected as lots of graph neural networks only work with undirected graphs. The graph is created with the PyTorch-Geometric library. The model MetaPath2Vec [4] is trained on the graph in a self-supervised way to create shallow embeddings for the source IP nodes. It trains by setting a random walk starting from a node and going to another node from an existing edge, and generate these sequences. These sequences are then trained to predict its context nodes (nodes surrounding left and right of the nodes). The output is embedding vectors for each of the data. The model is trained for 5 epochs with a batch size of 256.

The graph features are described below. **Source Node Features**

- Number of destination ip addresses scanned
- Approximate number of flows: this should be the number of observed unique (destination_ip, source_port, protocol) pairs
- Number of packets sent
- Average packet size

Nealson Setiawan
nealsons@usc.edu

Protocol Used	Number of flows
TCP	109596
UDP	21136
ICMP	2224

Table 1: Number of protocols used

- Number of B {B | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {C | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {D | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- Number of /24 blocks scanned
- Flag on whether the protocol used is TCP
- Flag on whether the protocol used is UDP
  Destination Port Features
- Destination Port number

In Table 1, we can see that there are only 3 protocols in the dataset. Instead of making a different type of edge for each type of protocol, a simple flag is used to save memory issues as it kept crashing Colab with out-of-memory issues.

Clustering Initial clustering in Project B consists of HDBSCAN [5] similar to DBSCAN employed in paper [2], but after exploring different clustering methods like HDBSCAN, KMeans, and Spectral clustering and optimizing each clustering method using Bayesian hyperparameter tuning on silhouette score, KMeans(K=7) was employed at the end for the highest silhouette score.

Silhouette score is described below: For each instance i:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}$$

where

- $a(i)$ is the avg distance from point i to all other points in its own cluster
- $b(i)$ is the minimum avg distance from point i to other points in any other cluster

Thus, the average silhouette score represents the "goodness-of-fit" of clusters.

6.4 Results

Before analyzing the data, an extra feature was made called "number of conficker destinations" scanned from subtracting the "number of destinations" scanned with "number of non-conficker destinations" scanned. By doing so, we can see if we can identify a cluster where conficker is the majority.

Initially, histograms were used to see the distributions of each feature. But the features proved too skewed, that even when logged still looked messy and unorganized. Thus, boxplots were used as a cleaner alternative or support of any claims made from this paper.

From the figures on this section, the analysis will hopefully shed some light into the breadth, granularity, and type of scanners.

The rest of the vizualizations will be on the Vizualization link at the end of the paper.

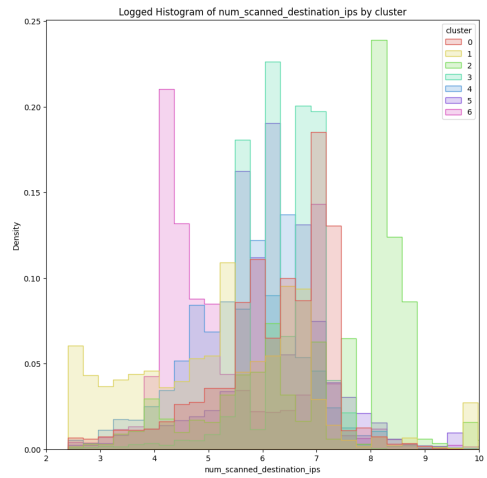

Figure 1: Logged histogram of number of scanned destination ips by cluster

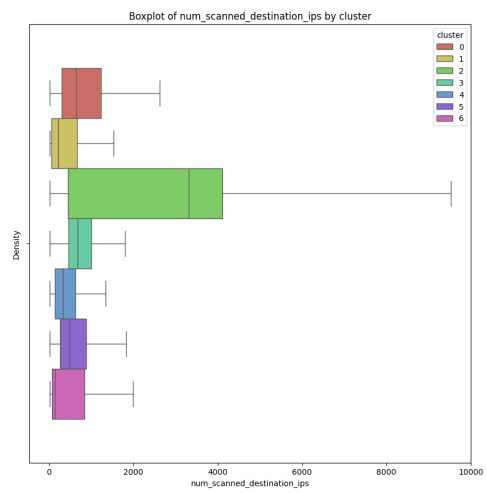

Figure 2: Boxplot of number of scanned destination IPs

6.5 Analysis

From the figures above, while there are many other types of scanners, the figures discovers four types of scanners in darknet data (these are my names that I came up with):

(1) Traditional horizontal scanners
(2) Subnet-based scanners
(3) UDP scanners
(4) Conficker scanners

Since all the scanners in the data are horizontal, as told by CAIDA in the dataset description, we can attempt to infer the horizontal scanners strategy. Furthermore, we also found a scanner that targets conficker destinations most, implying that it's a conficker worm. Note that the clusters can belong to multiple categories.

First, traditional horizontal scanners are scanners that tend to scan with low packet rates but high breadth of IPs targeted. This is

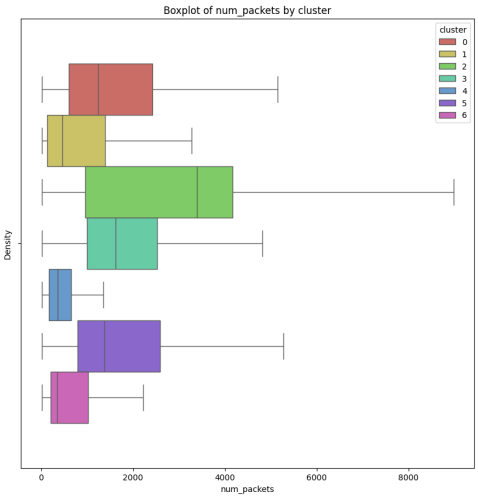

Figure 3: Boxplot of number of packets sent per cluster

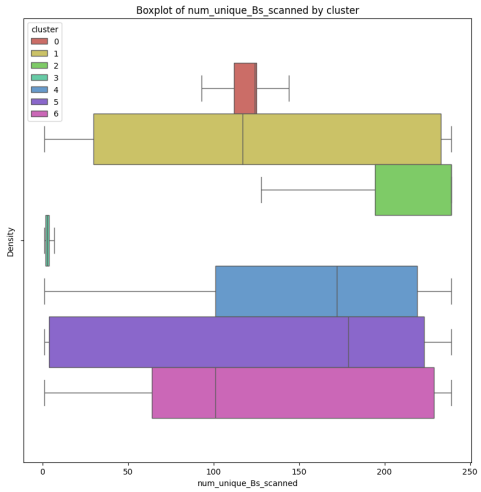

Figure 5: Boxplot of Number of unique Bs scanned by cluster

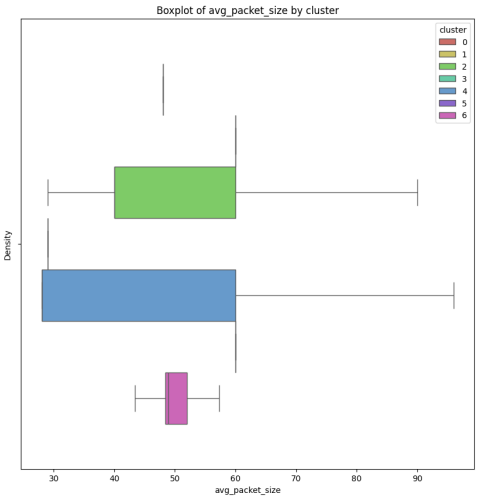

Figure 4: Boxplot of average packet size

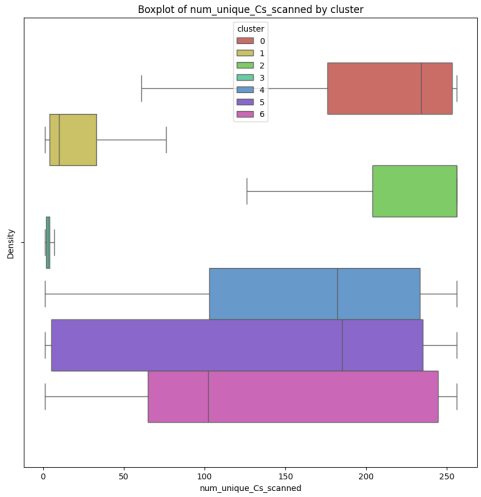

Figure 6: Boxplot of Number of unique Cs scanned by cluster

most apparent from figure 1, 2, 3, and 4. Where you can see that cluster 4, and 5 tend to send low-medium number of packets per destination. This implies that a more stealthy approach is taken to avoid detection, as typical of normal scanners. Additionally, you can see that these scanners target all octets of the darknet data. Since the network telescope captures \8 network, the traffic can target any of the other 24 octets. Thus, from seeing which octet is being targeted, we can infer if they focused on octets or not. Focusing on octet could mean that they tend to target subnets after a specific network portion is found to have a service. From figure 5,6,7, we can see that cluster 4,5 tend to scan each octet equally. Thus, could mean that they're scanning randomly to find services.

Second, subnet-based scanners focuses on finding specific subnets within a network. As described before, we can see this from figure 5,6,7, and this is most apparent for cluster 0, 1, and 3 where it tends to scan the C (third octet), B (second octet), and D (fourth octet) in an IPv4 network. This could imply that these scanners have found previous network portions that have services and could be scanning to find other services within a network.

Third, as seen in the protocol count in Table 2, there are scanners that tend to use UDP. The cluster in question is cluster 3. From figure 7, we can infer that these UDP scanners tend to target the fourth octet of an IP. This could imply that these scanners use UDP packets to target specific services discovery in the host portion after a service has been discovered at a certain network. In a \24 network, the D (fourth octet) would be the host portion of the IP. Also, UDP could potentially mean that these scanners are trying to target IoT devices because a lot of IoT device uses UDP-based connection to minimize latency.

Finally, without training on the amount of conficker destinations scanned, the methodology suggests that cluster 2 might be a conficker worm. As seen in figure 9, there is a high number of conficker

Nealson Setiawan
nealsons@usc.edu

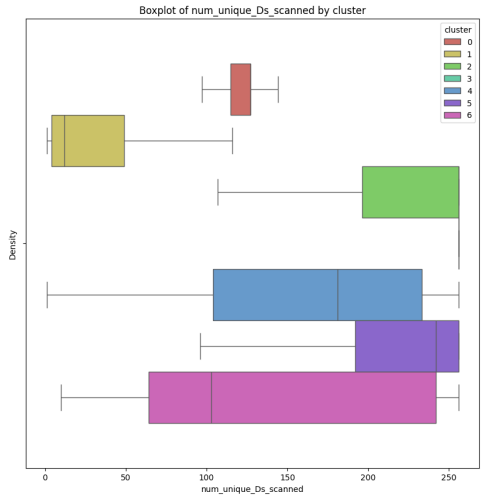

Figure 7: Boxplot of Number of unique Cs scanned by cluster

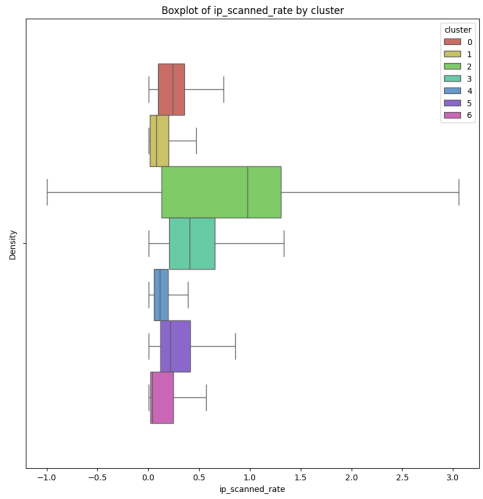

Figure 8: Boxplot of number of IP scanned rate

destinations scanned. This cluster tend to use a mix of UDP and TCP as seen in Table 2, and scan the highest number of destination IPs and number of packets from figure 1,2,3. This is typical of a worm-like behavior and provides strong evidence for this claim.

7 FUTURE WORK AND CONCLUSION

Hopefully this paper would bring insight into the types of scanners found in darknet data along with introducing a new novel way to represent darknet data as a graph. Representing darknet netflows as a graph proves to provide good separation in the data, and allow for scanner behavior to be discovered effectively.

More future work needs to be done on applying this methodology on Mirai attacks (September 14th 2016), as this would offer more credibility to this approach. Furthermore, applying a temporal metric similar to paper [7] could also allow for future attacks to be alarmed. Finally, comparing this method to a DarkVec's ground truth (if found), can be useful to determine its performance towards other embedding-cluster type systems.

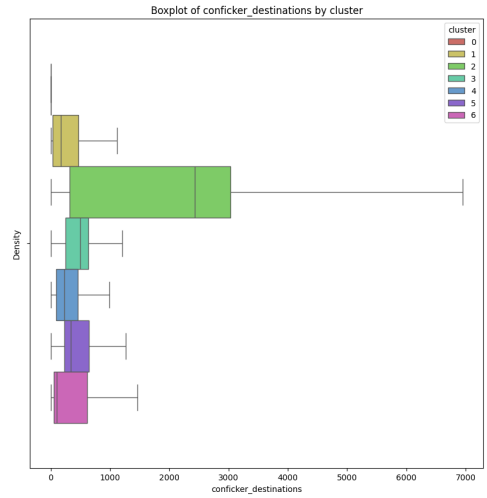

Figure 9: Boxplot of number of conficker destinations scanned

Table 2: Protocol Count by Cluster

cluster	protocol	count
0	6	12897
0	1	0
0	17	0
1	6	3582
1	17	18
1	1	0
2	6	7090
2	17	1492
2	1	0
3	17	19618
3	1	0
3	6	0
4	1	2224
4	6	6
4	17	2
5	6	83032
5	1	0
5	17	0
6	6	2989
6	17	6
6	1	0

8 BIBLIOGRAPHY

[1] M. Kallitsis, R. Prajapati, V. Honavar, D. Wu and J. Yen, "Detecting and Interpreting Changes in Scanning Behavior in Large Network Telescopes," in IEEE Transactions on Information Forensics and Security, vol. 17, pp. 3611-3625, 2022, doi: 10.1109/TIFS.2022.3211644

[2] S. Torabi, E. Bou-Harb, C. Assi, E. B. Karbab, A. Boukhtouta and M. Debbabi, "Inferring and Investigating IoT-Generated Scanning Campaigns Targeting a Large Network Telescope," in IEEE Transactions on Dependable and Secure Computing, vol. 19, no. 1, pp. 402-418, 1 Jan.-Feb. 2022, doi: 10.1109/TDSC.2020.2979183.

[3] Luca Gioacchini, Luca Vassio, Marco Mellia, Idilio Drago, Zied Ben Houidi, and Dario Rossi. 2021. DarkVec: automatic analysis of darknet traffic with word embeddings. In Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21). Association for Computing Machinery, New York, NY, USA, 76–89. https://doi.org/10.1145/3485983.3494863

[4] Yuxiao Dong, Nitesh V. Chawla, and Ananthram Swami. 2017. Metapath2vec: Scalable Representation Learning for Heterogeneous Networks. In Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '17). Association for Computing Machinery, New York, NY, USA, 135–144. https://doi.org/10.1145/3097983.3098036

[5] C. Malzer and M. Baum, "A Hybrid Approach To Hierarchical Density-based Cluster Selection," 2020 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI), Karlsruhe, Germany, 2020, pp. 223-228, doi: 10.1109/MFI49285.2020.9235263.

[6] The CAIDA UCSD Network Telescope "Darknet Scanners" Dataset , https://www.caida.org/catalog/datasets/telescope-darknet-scanners_dataset/

[7] Rupesh Prajapati, Vasant Honavar, Dinghao Wu, John Yen, and Michalis Kallitsis. 2021. Shedding light into the darknet: scanning characterization and detection of temporal changes. In Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21). Association for Computing Machinery, New York, NY, USA, 469–470. https://doi.org/10.1145/3485983.3493347

9 CODE

Code: https://drive.google.com/drive/folders/1kVSC8jVMxbLxfSRKWZs4fuL9-EGkdvur?usp=sharing Vizualizations: https://drive.google.com/drive/folders/1Qp1So0hsYdIuQT3a2Ao2Fohj8DgkV7NQ?usp=sharing